PennyPilot

Smart Budgeting & Savings Companion

PROJECT ONE | FLUTTER MOBILE APPLICATION | PROPOSAL — OCTOBER 18, 2026

Team members: Akshitha Sainath Sanagarapu • Mouni Sri Nallapaneni

1. The idea

Why a mobile app?

The phone is usually where a spending decision happens: at a café counter, on the bus, or while shopping. Instead of asking someone to remember purchases until they get home, PennyPilot is designed around quick entries and a useful summary the next time they open the app. The aim is not to replace a bank or offer financial advice. It is to make everyday spending easier to notice and understand.

Keeping track of small purchases is easy to put off. By the end of the week, it can be hard to remember where the money went or how much is still available for groceries, transport, and other essentials. We want to build a simple mobile app that makes these everyday decisions easier without asking users to connect bank accounts or share financial information online.

PennyPilot will help users record income and expenses, organize transactions, set category budgets, and follow a short savings challenge. A local spending coach will give understandable suggestions based on saved transactions. The app is designed to work entirely offline, with data stored on the device.

2. Goals and users

A typical user journey

A student sets a monthly budget, creates categories such as Food and Transport, and records a few purchases during the week. On the dashboard, they can see how much of the budget is left. When they want more context, they open History & Insights, filter the entries, and review a simple chart. If a record is wrong, they can edit it rather than starting over.

- Make it quick to record and correct everyday transactions.
- Show clear progress toward a budget or savings target.
- Help users notice spending patterns through local, explainable suggestions.
- Keep records and preferences available after closing and reopening the app.

Our intended users are students and other people who want a lightweight way to manage day-to-day spending. We are intentionally keeping this different from a full financial forecasting platform: the emphasis is on mobile interactions, habits, and reliable offline use.

3. Proposed features

What we will prioritize

We will build the essentials first: reliable transaction and category management, an understandable budget summary, and persistence across restarts. The savings challenge, reminders, and data export are planned enhancements. If time becomes tight, we will reduce optional polish rather than leave the required CRUD, workflow, testing, or offline behavior unfinished.

Feature	What the user can do

Transactions and categories	Add, view, edit, and delete income/expense records; create and manage custom categories.
Budget planner	Set a category spending limit, see used/remaining amounts, and adjust a limit with a slider.
Savings challenge	Start a seven-day challenge, record contributions, view progress, and finish or reset it.
History and insights	Filter records by category or date and view total expenses and a category breakdown.
Local spending coach	Compare recent spending with a simple baseline and show one practical, rule-based suggestion.
Reminders and preferences	Choose notification preferences and receive locally scheduled reminders; settings persist.

4. Five main screen roles

How the screens connect

The dashboard acts as the starting point. A user can open the editor to add a transaction, tap an existing item to inspect its details, or move to History & Insights to look for patterns. Settings holds budget preferences and reminder controls. Category management will be accessible from the entry flow so the two related SQLite entities are genuinely editable, not just stored behind the scenes.

Screen	Purpose and key interaction
Home / Dashboard	Shows this month’s totals, budget progress, current challenge, and shortcuts.
Editor	Form to add or edit a transaction or category, with required-field and amount validation.
Detail / Activity	Shows an individual transaction and actions to edit or delete it; also shows challenge activity.
History / Insights	Search/filter transaction history, view totals, and read the spending coach’s suggestion.
Settings	Set currency/display preferences, notification options, and manage categories.

Budget and challenge controls can be opened from the dashboard or detail view rather than requiring extra top-level navigation tabs.

5. Navigation and screen wireframes

Design direction

We chose a calm teal-and-cream palette, rounded cards, clear labels, and a small number of actions per screen. The sample amounts and dates in the mockups are illustrative only. The screens are not built yet; they communicate our intended layout and help us discuss usability before development. We expect to revise them after peer feedback.

The following diagrams show the proposed flow and layout. Labels and visual details may change after peer feedback and instructor approval.

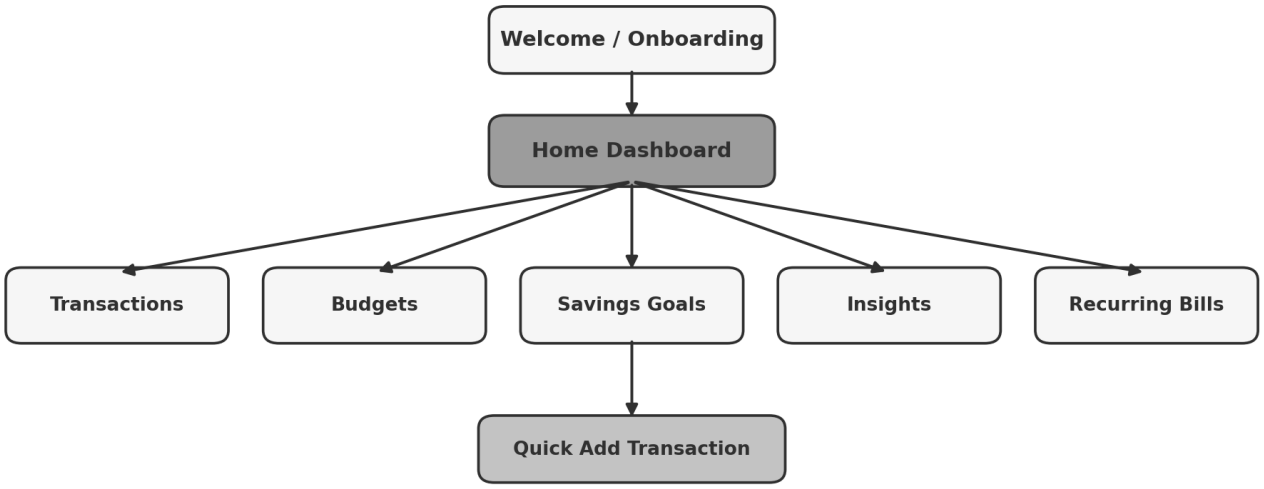


Figure 1. Proposed navigation between the main areas of the app.

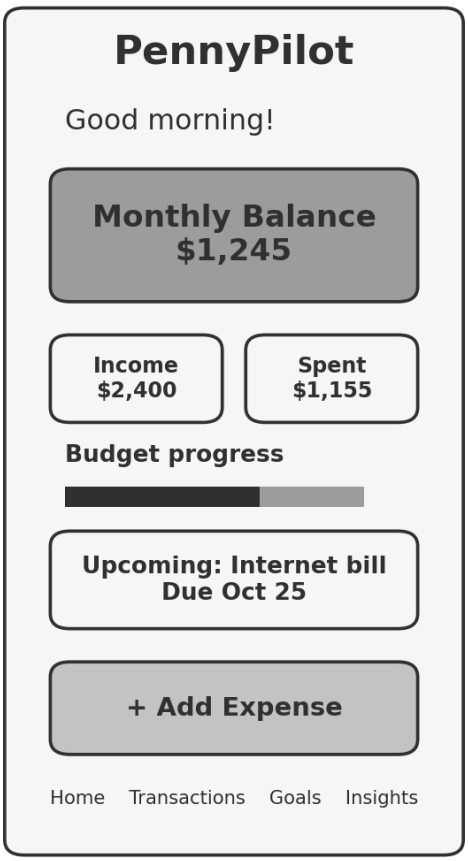
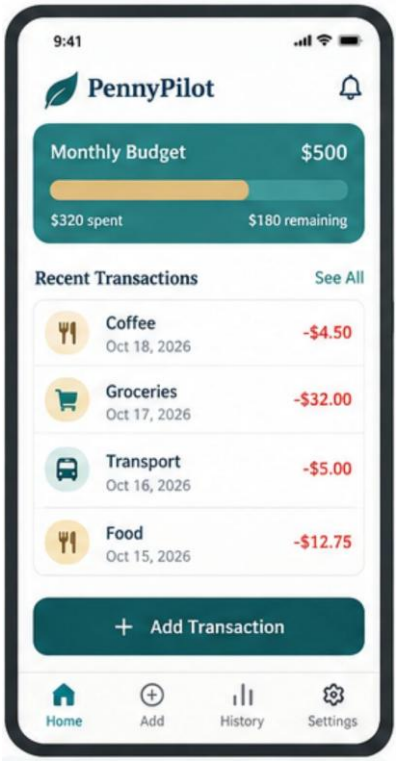


Figure 2. Early dashboard wireframe: balance summary, budget progress, quick entry, and bottom navigation.

Proposed screen mockups

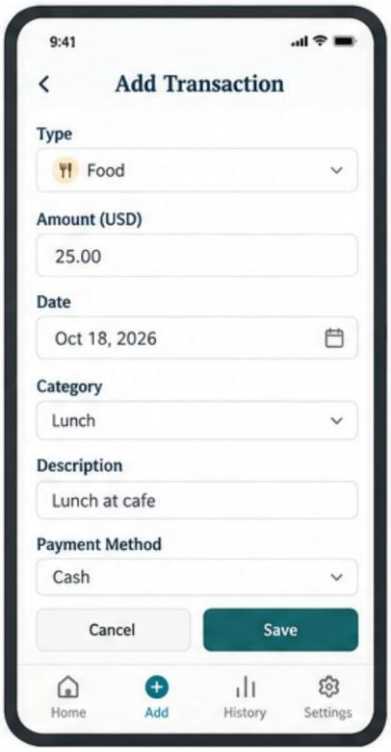
The following five screens are proposed designs, not screenshots of a working application. They illustrate the intended user flow and will be refined after instructor approval and peer feedback.

Screen 1: Dashboard (Home)



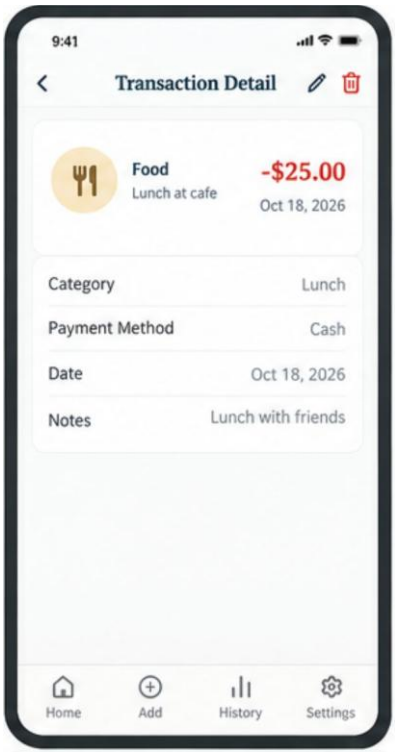
The dashboard gives a quick view of the monthly budget, the amount spent, and what remains. Recent transactions appear below the summary, with a clear button to add a new entry.

Screen 2: Add / Edit Transaction



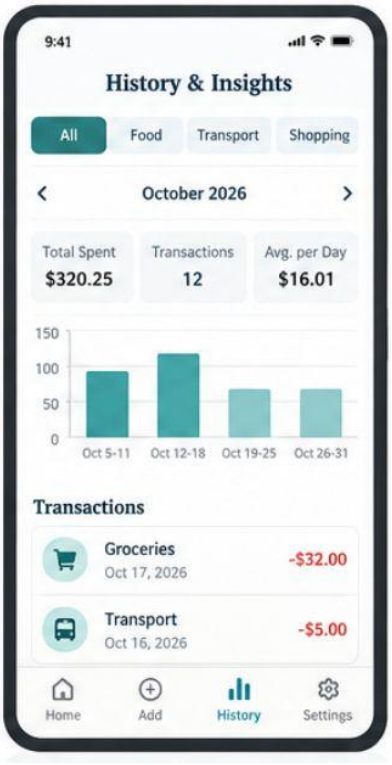
This form allows users to record or correct an expense. It includes an amount, date, category, description, and payment method. Required fields and amounts will be validated before saving.

Screen 3: Transaction Detail



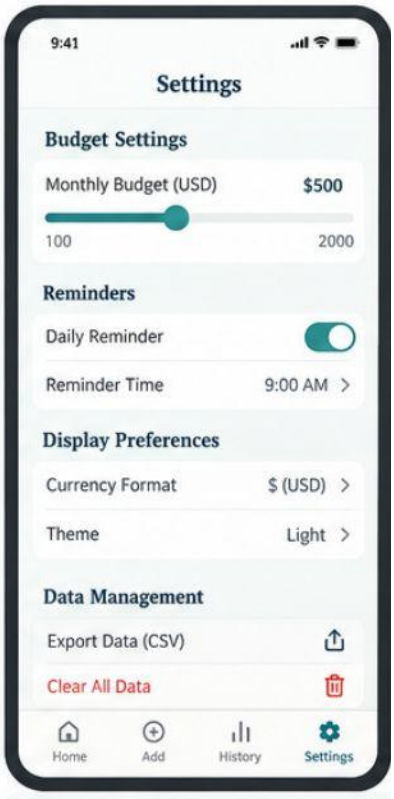
Selecting a transaction opens its details. Users can review the recorded information, edit mistakes, or delete the entry after a confirmation prompt.

Screen 4: History & Insights



This screen groups spending by period and category. It provides totals, transaction counts, and a simple chart to help users understand where their money goes.

Screen 5: Settings



Settings will include a monthly-budget slider, local reminder preferences, display choices, and data-management actions. Preferences will persist when the app restarts.

6. Important interaction details

Making mistakes recoverable

For financial records, a small mistake can change a total noticeably. We therefore plan to validate amounts before saving, allow edits from the detail screen, and confirm destructive actions. Deleting a category that still has transactions will be blocked with a helpful explanation, so historical records do not lose their meaning.

- Empty state: a new user sees a friendly prompt to add a first category and transaction.
- Validation: amounts must be greater than zero; required fields and invalid dates show clear messages.
- Deletion: a confirmation dialog appears before removing a saved record.
- Offline/restart: saved transactions, challenge progress, and preferences remain available.
- Accessibility: readable contrast, labeled controls, and touch targets large enough for mobile use.

7. Data design and technical approach

Data consistency and restart behavior

Each transaction belongs to one category through a category ID. Both tables will have create, read, update, and delete paths, including empty-state and error handling. Transactions will remain on the device in SQLite; smaller preferences, such as the monthly budget and reminder choice, will be stored separately. We will verify the saved data by closing and reopening the app rather than assuming a successful save is enough.

The required related SQLite entities will be Category and Transaction. Each will support validated create, read, update, and delete operations. A transaction belongs to one category; a category may have many transactions. We will prevent deletion of a category while transactions still reference it, and explain how to reassign or remove those transactions first.

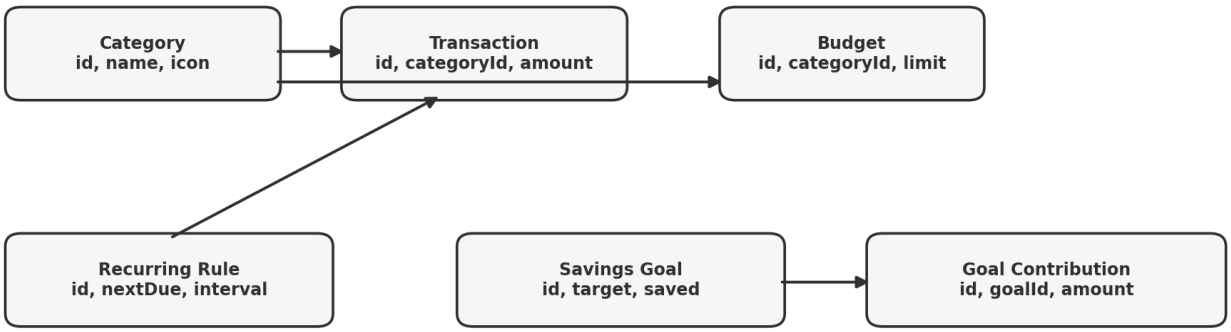


Figure 3. Proposed data model. Category and Transaction are the two required related CRUD entities; additional tables support optional budgeting and challenge functionality.

For the core build, Category stores an ID, name and icon; Transaction stores an ID, category ID, type, amount, date and note. Budget and challenge records can be added after the required CRUD flows are stable.

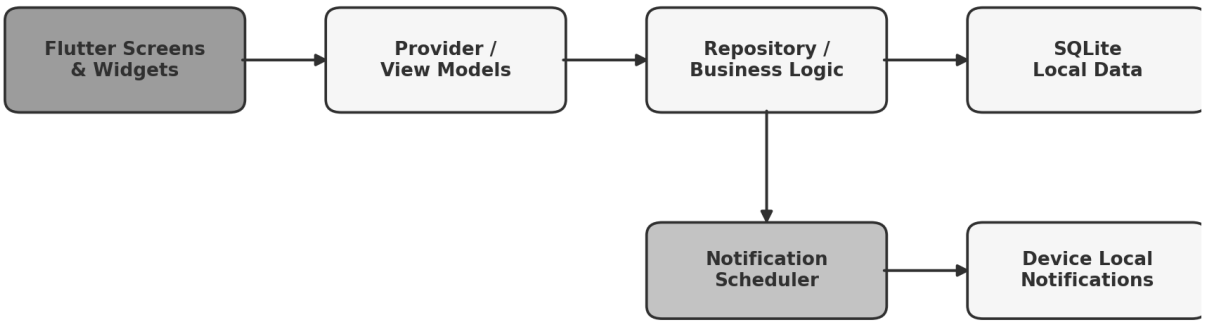


Figure 4. Planned offline architecture. No Firebase, cloud authentication, remote database, or network service is required.

Implementation choices

- Flutter and Dart for screens, navigation, reusable StatelessWidget and StatefulWidget components, forms, lists, dialogs, and gestures.
- SQLite through sqflite for local records; a repository layer separates database operations from the UI.
- Provider / ChangeNotifier for shared state, with explicit loading, success, and error states.
- SharedPreferences for lightweight settings; flutter_local_notifications for reminders if approved and supported on test devices.
- A local chart package for category totals, with no external financial APIs or bank integration.

8. Required local smart helper

An example of an explainable suggestion

Suppose a user has spent $85 of a $100 Food budget. The helper could show: “You have used 85% of your Food budget. Consider checking this category before your next purchase.” We will document the threshold, inputs, and reason for the message. Users will be able to ignore the suggestion; it will not modify their transactions or make decisions on their behalf.

How we will evaluate it

We will prepare examples around thresholds, missing budgets, zero spending, and category changes. For each case, we will record the expected message, the actual message, and whether it was useful or misleading. We will compare these results with a fixed generic reminder and keep a counterexample where our rule needs improvement. This is a rule-based helper, not a trained machine-learning model.

Our proposed helper is a “Spending Check.” It will examine the current week’s expenses and give a short suggestion such as “Food spending is close to your weekly limit.” The logic will be transparent: compare spending against the user’s budget and the previous week’s total. It is a rule-based helper, not a trained AI model.

Baseline: always show the same generic tip (“Review your expenses regularly”). Evaluation: create a small set of test cases covering under-budget, near-limit, over-limit, and insufficient-history situations. We will compare how often the rule-based helper gives the expected message versus the baseline, and document limitations and incorrect or unhelpful outputs in AI_FEATURE.md.

9. Development and testing plan

How we will test together

Both teammates will use a small shared dataset with known totals, then test create/edit/delete behavior independently. We will check navigation and validation with a second person, record any confusing interaction in ClickUp, and link the fix to a commit or test. We will also test on two devices or emulators in airplane mode, including a full app restart, so "offline-first" is demonstrated rather than simply stated.

Date / checkpoint	Planned work and evidence
Oct 18 — proposal	Submit Word proposal, signed attachment, ClickUp board URL and dated board snapshot; wait for approval.
Oct 19–21 — UI	After approval: Flutter structure, five screen roles, navigation, first CRUD vertical slice; Oct 21 board snapshot.
Oct 22–24 — core	Complete both entities' CRUD, budget and challenge flows, local helper, persistence; Oct 24 snapshot.
Oct 25–26 — verify	Unit/widget tests, restart and offline checks on two devices/emulators, peer feedback and a documented UI revision.
Oct 27–28 — deliver	Prepare team showcase, individual evidence, report, APK and final Oct 28 board snapshot; submit by 11:59 PM ET.

What we will test

- Both Category and Transaction: create, list, edit, delete, invalid input, and confirmation behavior.
- Parent/child integrity: category deletion is blocked when linked transactions exist.
- Challenge: start, add progress, finish, reset, and verify restart persistence.
- Filtering and totals: expected calculations match manually prepared test data.
- Spending Check: test expected suggestions against the generic-tip baseline.
- Graduate-level work: state-management behavior, repository separation, notification handling, and automated unit/widget tests.

10. Team coordination

Working agreement

We will keep tasks small enough to review, use ClickUp to show who owns each item and what "done" means, and open GitHub pull requests for changes that affect shared screens or data models. Each of us will work across design, implementation, and testing instead of treating the app as two disconnected halves. The assignment of individual features may change as we learn more during development.

Both members will take part in design decisions, coding, testing, reviews, and the final presentation. The split below is an initial plan; the ClickUp board will record the final ownership and evidence.

Akshitha	Mouni	Shared
Navigation, dashboard, insights, smart-helper evaluation	SQLite entities, CRUD screens, savings challenge, persistence tests	Wireframes, reviews, integration, notifications, documentation, demo

We have created a GitHub repository and use a shared ClickUp Kanban board with task owners, acceptance criteria, and links to commits/tests. The course requires at least 20 meaningful commits per person over the project; we will make regular, reviewable contributions rather than artificial commits.

11. Expected deliverables

Evidence we plan to preserve

Alongside the app and source code, we will keep screenshots of the four dated ClickUp checkpoints, a record of meaningful commits from both members, testing notes, a short architecture explanation, and a clear account of the smart helper's limitations. Our presentation will show real user flows and explain the decisions we made, including what changed after review.

After approval and development, we aim to deliver an installable Flutter APK, source repository, documented SQLite schema, automated test results, AI_FEATURE.md, a short project report, a team presentation/video, and individual evidence of contributions. For the proposal stage, we will submit the Word document and the required approval materials.

12. Why this project is worth building

What we hope to learn

The most valuable part of PennyPilot is not the appearance of the dashboard. It is making several small pieces work together: a usable form, dependable local storage, state that updates correctly, clear calculations, and helpful error messages. That combination makes the project a practical exercise in building a complete mobile experience within a short timeline.

PennyPilot is deliberately small enough to finish within the course timeline but still involves the parts of mobile development that matter: thoughtful screens, reliable data relationships, state management, offline persistence, local notifications, and testing. If it works well, it can also serve as a portfolio example of a complete Flutter application rather than only a visual prototype.

Project links and proposal submission attachments

GitHub repository URL: https://github.com/MouniSri039/PennyPilot

ClickUp board URL: https://app.clickup.com/90141780871/v/b/2kydwuw7-354

Signed group statement: Separate signed attachment to be submitted with this proposal.

MOBILE APPLICATION DEVELOPMENT / PROJECT ONE

Team Project Acknowledgment

Attach a completed, signed copy to the proposal and include it in the final package.

Course / section
Mobile App Development ACS67 Fall 2026

Team name
Team PennyPilot

Project title
PennyPilot – Smart Budgeting & Savings Companion

Acknowledgment of project requirements

By signing below, each listed member acknowledges the published Project One requirements and agrees to participate meaningfully throughout the group project.

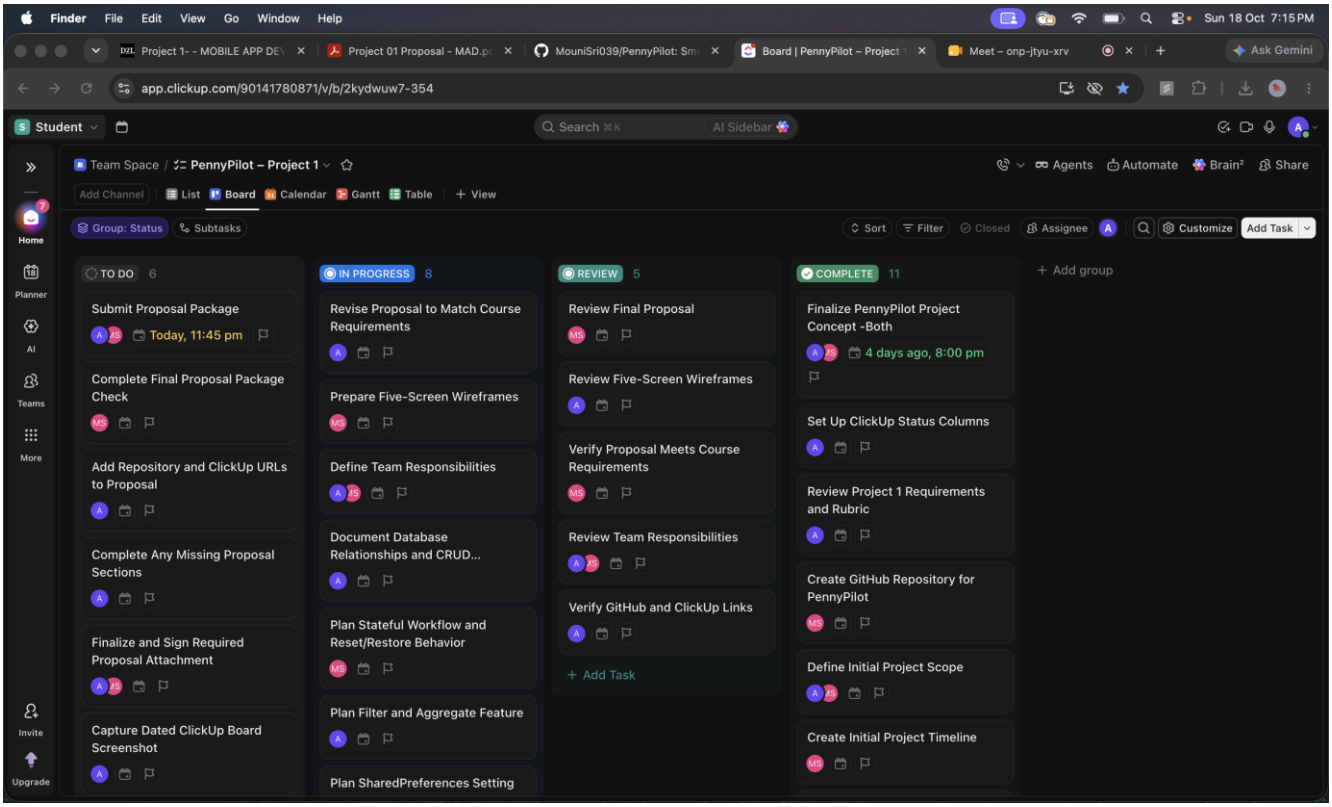
1. The proposal is due October 18, 2026, and the final project is due October 28, 2026, both at 11:59 PM in Eastern Time (America/New_York). Instructor-team approval is required before implementation.
2. The app uses SQLite CRUD for both related entities, SharedPreferences settings, and one required explainable local smart helper with pathway-specific evaluation. Rules qualify when labeled honestly. No cloud storage, backend, authentication, or remote inference.
3. Each member contributes authentic work, maintains steady meaningful commits and ClickUp task-to-build evidence, and can explain their contribution, code, and decisions. AI assistance, if used, must be disclosed and verified under the assignment guidance.
4. The team provides a 20-25 minute presentation with integrated curated Q&A and evidence of app behavior, code, testing, and individual ownership.
5. Assessment is individual; members may receive different grades. Each member uploads the required final package individually to the designated iCollege destination.

This acknowledgment summarizes the existing assignment; it does not add new grading policies or replace the full project requirements.

Member signatures

All participating members must sign and date. Use the continuation area if needed.

Member 1 full name	Student ID	Signature	Date
Akshitha Sainath Sanagarapu	003027780	Akshitha Sainath Sanagarapu	10/18/2026
Member 2 full name	**Student ID**	**Signature**	**Date**
Mouni Sri Nallapaneni	003013851	Mouni Sri Nallapaneni	10/18/2026
Member 3 full name	**Student ID**	**Signature**	**Date**


Finder File Edit View Go Window Help
Sun 18 Oct 7:15 PM
app.clickup.com/90141780871/v/b/2kydwuw7-354
Student
Search ⌘K
AI Sidebar
Team Space / PennyPilot – Project 1
Agents Automate Brain² Share
Add Channel List Board Calendar Gantt Table + View
Group: Status Subtasks
Sort Filter Closed Assignee Customize Add Task
Home Planner AI Teams More Invite Upgrade
TO DO 6
Submit Proposal Package
Today, 11:45 pm
Complete Final Proposal Package Check
Add Repository and ClickUp URLs to Proposal
Complete Any Missing Proposal Sections
Finalize and Sign Required Proposal Attachment
Capture Dated ClickUp Board Screenshot
IN PROGRESS 8
Revise Proposal to Match Course Requirements
Prepare Five-Screen Wireframes
Define Team Responsibilities
Document Database Relationships and CRUD...
Plan Stateful Workflow and Reset/Restore Behavior
Plan Filter and Aggregate Feature
Plan SharedPreferences Setting
REVIEW 5
Review Final Proposal
Review Five-Screen Wireframes
Verify Proposal Meets Course Requirements
Review Team Responsibilities
Verify GitHub and ClickUp Links
+ Add Task
COMPLETE 11
Finalize PennyPilot Project Concept -Both
4 days ago, 8:00 pm
Set Up ClickUp Status Columns
Review Project 1 Requirements and Rubric
Create GitHub Repository for PennyPilot
Define Initial Project Scope
Create Initial Project Timeline
+ Add group